

Basket Microservice

Code Explanation + Business Logic Documentation

C# .NET | Clean Architecture | CQRS | MediatR Pattern

Basket Microservice -

Basket () microservice .

- / :
- :
- :

Redis (Cache) :

- Redis (In-Memory)
- TTL ()

(Architecture)

```
Basket Service
|-- Basket.API          <-  - Controllers - HTTP Endpoints
|-- Basket.Application  <-  - Commands/Queries/Handlers/Mappers
|-- Basket.Core         <-  - Entities - Interfaces
\-- Basket.Infrastructure <-  - Redis Repository - Settings
```

Clean Architecture :

- Core layer
- Application Core
- Infrastructure Core interfaces
- API

Basket.Core -

>> Entities ()

ShoppingCart.cs

```
public class ShoppingCart
```

```
{
    public string UserName { get; set; }
    public List<ShoppingCartItem> Items { get; set; } = new List<ShoppingCartItem>();

    public ShoppingCart() { }

    public ShoppingCart(string userName)
    {
        UserName = userName;
    }
}
```

: "" .

- [UserName]: Redis
- [Items]:
- Constructor Deserialization Redis
- Constructor userName

ShoppingCartItem.cs

```
public class ShoppingCartItem
{
    public int Quantity { get; set; }
    public decimal Price { get; set; }
    public string ProductId { get; set; }
    public string ProductName { get; set; }
    public string ImageFile { get; set; }
}
```

: .

- [ProductId]: Catalog Service
- [Quantity]:
- [Price]: ()
- [ImageFile]:

>> Repository Interface

IBasketRepository.cs

```
public interface IBasketRepository
{
    Task<ShoppingCart> GetBasket(string userName);
    Task<ShoppingCart> UpSertBasket(ShoppingCart basket);
    Task DeleteBasket(string userName);
}
```

: (Contract) Repository Basket.

- [GetBasket]: Redis
- [UpSertBasket]: (Up = Update + Sert = Insert)
- [DeleteBasket]:

Basket.Infrastructure -

>> BasketRepository.cs

```
public class BasketRepository : IBasketRepository
{
    private readonly IDistributedCache _redisCache;

    public BasketRepository(IDistributedCache redisCache)
    {
        _redisCache = redisCache;
    }

    public async Task DeleteBasket(string userName)
    {
        await _redisCache.RemoveAsync(userName);
    }

    public async Task<ShoppingCart> GetBasket(string userName)
    {
        var basket = await _redisCache.GetStringAsync(userName);
        if (string.IsNullOrEmpty(basket))
            return null;
        return JsonConvert.DeserializeObject<ShoppingCart>(basket);
    }

    public async Task<ShoppingCart> UpSertBasket(ShoppingCart shoppingCart)
    {
        await _redisCache.SetStringAsync(
            shoppingCart.UserName,
            JsonConvert.SerializeObject(shoppingCart)
        );
        return await GetBasket(shoppingCart.UserName);
    }
}
```

- [IDistributedCache]: Abstraction Cache System (Redis, Memory, etc.)
- GetBasket: Redis JSON string Object
- UpSertBasket: Object JSON Redis Key = userName
- DeleteBasket: Key Redis

>> CacheSettings.cs

```
public class CacheSettings
{
    public string ConnectionString { get; set; }
}
```

: Redis [appsettings.json].

Basket.Application -

>> CQRS Pattern

CQRS (Command Query Responsibility Segregation):

- Commands: (//)
 - Queries:
-

>> Commands ()

CreateShoppingCartCommand.cs

```
public record CreateShoppingCartCommand(  
    string UserName,  
    List<CreateShoppingCartItemDto> Items  
) : IRequest<ShoppingCartResponse>;
```

: .

- [record]: immutable () Commands
- [IRequest<ShoppingCartResponse>]: MediatR ShoppingCartResponse

DeleteBasketByUserNameCommand.cs

```
public record DeleteBasketByUserNameCommand(string UserName) : IRequest<Unit>;
```

: .

- [IRequest<Unit>]: Unit = void MediatR
-

>> Queries ()

GetBasketByUserNameQuery.cs

```
public record GetBasketByUserNameQuery(string UserName) : IRequest<ShoppingCartResponse>;
```

: . .

>> Handlers ()

GetBasketByUserNameHandler.cs

```
public class GetBasketByUserNameHandler  
    : IRequestHandler<GetBasketByUserNameQuery, ShoppingCartResponse>  
{  
    private readonly IBasketRepository _basketRepository;  
  
    public GetBasketByUserNameHandler(IBasketRepository basketRepository)
```

```
{
    _basketRepository = basketRepository;
}

public async Task<ShoppingCartResponse> Handle(
    GetBasketByUserNameQuery request,
    CancellationToken cancellationToken)
{
    var shoppingCart = await _basketRepository.GetBasket(request.UserName);

    if (shoppingCart == null)
    {
        return new ShoppingCartResponse(request.UserName)
        {
            Items = new List<ShoppingCartItemResponse>()
        };
    }

    return shoppingCart.ToResponseUsingDelegate();
}
}
```

1. Query MediatR
2. Repository
3. -> (null error)
4. -> Response Delegate Mapper

CreateShoppingCartHandler.cs

```
public class CreateShoppingCartHandler
    : IRequestHandler<CreateShoppingCartCommand, ShoppingCartResponse>
{
    private readonly IBasketRepository _basketRepository;

    public CreateShoppingCartHandler(IBasketRepository basketRepository)
    {
        _basketRepository = basketRepository;
    }

    public async Task<ShoppingCartResponse> Handle(
        CreateShoppingCartCommand request,
        CancellationToken cancellationToken)
    {
        // Command Entity
        var shoppingCartEntity = request.ToEntity();

        // Redis
        var updatedCart = await _basketRepository.UpSertBasket(shoppingCartEntity);

        // Response
        return updatedCart.ToResponse();
    }
}
```

1. Command -> Entity: API
2. UpSert in Redis: Redis
3. Entity -> Response: Redis API

DeleteBasketByUserNameHandler.cs

```
public class DeleteBasketByUserNameHandler
    : IRequestHandler<DeleteBasketByUserNameCommand, Unit>
{
    private readonly IBasketRepository _basketRepository;

    public DeleteBasketByUserNameHandler(IBasketRepository basketRepository)
    {
        _basketRepository = basketRepository;
    }

    public async Task<Unit> Handle(
        DeleteBasketByUserNameCommand request,
        CancellationToken cancellationTokentoken)
    {
        await _basketRepository.DeleteBasket(request.UserName);
        return Unit.Value;
    }
}
```

: Handler . Redis Unit.Value .

>> DTOs (Data Transfer Objects)

ShoppingCartDto.cs

```
public record ShoppingCartDto(string userName, List<ShoppingCartItemDto> Items);

public record ShoppingCartItemDto(
    string ProductId,
    string ProductName,
    string ImageFile,
    decimal Price,
    int Quantity);

public record CreateShoppingCartItemDto(
    string ProductId,
    string ProductName,
    string ImageFile,
    decimal Price,
    int Quantity);
```

: DTOs / Client (Frontend Mobile App).

- [ShoppingCartDto]:
 - [ShoppingCartItemDto]:
 - [CreateShoppingCartItemDto]:
-

>> Responses

ShoppingCartResponse.cs

```
public record class ShoppingCartResponse
{
    public string UserName { get; init; }
    public List<ShoppingCartItemResponse> Items { get; init; }

    public ShoppingCartResponse() { ... }
    public ShoppingCartResponse(string userName) { ... }
    public ShoppingCartResponse(string userName, List<ShoppingCartItemResponse> items) { ... }

    public decimal TotalPrice => Items.Sum(item => item.Price * item.Quantity);
}
```

- API
 - [TotalPrice]:
 - [init]:
-

>> Mappers

BasketMapper.cs

```
public static class BasketMapper
{
    // : Extension Method
    public static ShoppingCartResponse ToResponse(this ShoppingCart shoppingCart)
    {
        return new ShoppingCartResponse
        {
            UserName = shoppingCart.UserName,
            Items = shoppingCart.Items.Select(item => new ShoppingCartItemResponse
            {
                Quantity = item.Quantity,
                ImageFile = item.ImageFile,
                Price = item.Price,
                ProductId = item.ProductId,
                ProductName = item.ProductName
            }).ToList()
        };
    }

    // : Delegate-Based Mapper
    public static ShoppingCartResponse ToResponseUsingDelegate(this ShoppingCart cart) =>
        MapCart(cart);

    public static readonly Func<ShoppingCart, ShoppingCartResponse> MapCart =
        cart => new ShoppingCartResponse { ... };

    // Command -> Entity Mapper
    public static ShoppingCart ToEntity(this CreateShoppingCartCommand command)
```

```
{
    return new ShoppingCart
    {
        UserName = command.UserName,
        Items = command.Items.Select(item => new ShoppingCartItem { ... }).ToList()
    };
}
```

: :

1. [ToResponse]: Entity -> Response ()
 2. [ToResponseUsingDelegate]: Func Delegate ()
 3. [ToEntity]: Command -> Entity Redis
-

Basket.API -

>> BasketController.cs

```
public class BasketController : Controller
{
    private readonly IMediator _mediator;

    public BasketController(IMediator mediator)
    {
        _mediator = mediator;
    }

    // GET: api/v1/basket/{userName}
    [HttpGet("{userName}")]
    public async Task<ActionResult<ShoppingCartDto>> GetBasket(string userName)
    {
        var query = new GetBasketByUserNameQuery(userName);
        var result = await _mediator.Send(query);
        return Ok(result);
    }

    // POST: api/v1/basket
    [HttpPost]
    public async Task<ActionResult<ShoppingCartDto>> CreateOrUpdateBasket(
        [FromBody] CreateShoppingCartCommand command)
    {
        var result = await _mediator.Send(command);
        return Ok(result);
    }

    // DELETE: api/v1/basket/{userName}
    [HttpDelete("{userName}")]
    public async Task<IActionResult> DeleteBasket(string userName)
    {
        var cmd = new DeleteBasketByUserNameCommand(userName);
        await _mediator.Send(cmd);
    }
}
```



```
        return Ok();  
    }  
}
```

- API Endpoints:

Method	URL	
GET	/api/v1/basket/{userNa	
POST	/api/v1/basket	
DELETE	/api/v1/basket/{userName}	

>> Program.cs -

```
// Repository  DI Container  
builder.Services.AddScoped<IBasketRepository, BasketRepository>();  
  
// MediatR  
builder.Services.AddMediatR(cfg => cfg.RegisterServicesFromAssemblies(assemblies));  
  
// Redis  
builder.Services.Configure<CacheSettings>(  
    builder.Configuration.GetSection("CacheSettings"));
```

- [AddScoped]: instance Repository HTTP Request
- [AddMediatR]: Handlers
- [Configure<CacheSettings>]: appsettings.json CacheSettings class

(Data Flow)

>> / (POST)

```
Client  
| POST /api/v1/basket  
| Body: { userName, items: [...] }  
  
BasketController.CreateOrUpdateBasket()  
| CreateShoppingCartCommand  
| MediatR  
  
CreateShoppingCartHandler.Handle()  
| request.ToEntity() -> ShoppingCart Entity  
| _basketRepository.UpSertBasket(entity)  
  
BasketRepository.UpSertBasket()  
| JsonConvert.SerializeObject(cart) -> JSON  
| Redis.SetStringAsync(userName, json)  
| Redis  
  
CreateShoppingCartHandler  
| updatedCart.ToResponse() -> ShoppingCartResponse
```

```
Client <- ShoppingCartResponse ( TotalPrice )
```

>> (GET)

```
Client
| GET /api/v1/basket/ahmed

BasketController.GetBasket("ahmed")
| GetBasketByUserNameQuery("ahmed")

GetBasketByUserNameHandler.Handle()
| _basketRepository.GetBasket("ahmed")

BasketRepository.GetBasket()
| Redis.GetStringAsync("ahmed") -> JSON String
| JsonConvert.Deserialize -> ShoppingCart

GetBasketByUserNameHandler
| null ->
| -> ToResponseUsingDelegate()

Client <- ShoppingCartResponse
```

MediatR	CQRS Mediator Pattern
StackExchange.Redis	Redis
Newtonsoft.Json	Objects JSON
**Microsoft.Extensions.Ca	Abstraction layer Cache

Basket :

1. Redis

Catalog Service Frontend.